Layouts specified at the controller level support the `:only` and `:except` options. These options take either a method name, or an array of method names, corresponding to method names within the controller:

```
class ProductsController < ApplicationController
  layout "product", except: [:index, :rss]
end
```

With this declaration, the `product` layout would be used for everything but the `rss` and `index`methods.

2.2.14.4 Layout Inheritance

Layout declarations cascade downward in the hierarchy, and more specific layout declarations always override more general ones. For example:

- application_controller.rb
```
class ApplicationController < ActionController::Base
  layout "main"
end
```
- articles_controller.rb
```
class ArticlesController < ApplicationController
end
```
- special_articles_controller.rb
```
class SpecialArticlesController < ArticlesController
  layout "special"
end
```
- old_articles_controller.rb
```
class OldArticlesController < SpecialArticlesController
  layout false

  def show
    @article = Article.find(params[:id])
  end

  def index
    @old_articles = Article.older
    render layout: "old"
  end
  # ...
end
```

In this application:

- In general, views will be rendered in the `main` layout
- `ArticlesController#index` will use the `main` layout
- `SpecialArticlesController#index` will use the `special` layout
- `OldArticlesController#show` will use no layout at all
- `OldArticlesController#index` will use the `old` layout

2.2.14.5 Template Inheritance

Similar to the Layout Inheritance logic, if a template or partial is not found in the conventional path, the controller will look for a template or partial to render in its inheritance chain. For example:

```
# in app/controllers/application_controller
class ApplicationController < ActionController::Base
end

# in app/controllers/admin_controller
class AdminController < ApplicationController
end
```